EE465 Endsem Cheatsheet

Sources covered: syllabus PDFs in ee465, excluding `Introduction_to_Modern_Cryptography.pdf` per instruction: Bitcoin intro/tx/ECC/mining, hash/group theory, blockchain notes, Ethereum intro/data/tx/EVM/blocks/PoS, Solidity, JavaScript/Node, Tornado Cash, circom, Monero, Tendermint, consensus, quiz sheet.

1 Quick Find and Exam Moves

Find fast: EC formulas → Group Theory and Bitcoin ECC; ECDSA nonce/recovery → ECC + Worked Examples; RLP/HP/MPT → Ethereum Encoding; gas/tx types → Quick Tables; Solidity bugs → Solidity + Exam Templates; Tornado/Groth16 → Tornado Cash and zk; circom/R1CS → circom and R1CS; Monero/LSAG/Pedersen/range proofs → Monero; Dolev-Strong/FLP/FLM → Consensus; Tendermint phases → Tendermint; Casper/BLS/LMD-GHOST → Ethereum PoS.

Question verb	Fast response pattern
Compute	write formula, substitute numbers, reduce mod/bytes/gas each step
Trace	list state before, message/vote/action, state after, stopping condition
Prove/show	identify invariant, quorum intersection or hardness assumption, contradiction
Design	state inputs/outputs, constraints/checks, edge cases, gas/security note
Spot bug	name vulnerability, attack path, violated invariant, minimal fix

2 Bitcoin Basics

2.1 What Bitcoin Solves

Bitcoin = open-source cryptocurrency + decentralized network. Problems: counterfeiting, money creation rules, and double spending. Nodes verify; miners assemble blocks and do PoW; wallets hold keys; merchants wait for confirmations.

Workflow: Bob gives address out-of-band; Alice constructs transaction; broadcasts to P2P network; miners include in candidate block; once mined, confirmations accumulate as blocks build on top.

UTXO model: transaction consumes old unspent outputs and creates new outputs. Inputs reference (txid, $vout$) and provide unlocking data; outputs specify amount and locking script. Validity: referenced UTXOs exist, signatures/scripts pass, no double spend, sum inputs ≥ sum outputs; fee = input sum - output sum.

2.2 Keys, Addresses, Blocks

Bitcoin private key $d \in [1, n-1]$ for secp256k1; public key $Q = dG$. Address derivation: compressed/uncompressed public key → `SHA256256` → `RIPEMD160` = HASH160, add version byte, checksum = first 4 bytes of SHA256d, Base58Check encode. SegWit uses Bech32.

Block header fields: `nVersion` 4B, `hashPrevBlock` 32B, `hashMerkleRoot` 32B, `nTime` 4B, `nBits` 4B target encoding, `nNonce` 4B. Block hash = double SHA-256 of header. Merkle tree hashes txids pairwise; duplicate last hash if odd. Hash pointer chain gives tamper evidence.

Genesis block: first block hardcoded; embeds newspaper headline as timestamp/proof of no pre-mine style. **SPV**: light clients verify block headers + Merkle inclusion proof, not all scripts/state; trust longest/heaviest valid-header chain assumption.

2.3 P2P and Chain Selection

Node discovery via DNS seeds, peer exchange, address messages. Transactions propagate through mempool gossip; nodes check policy/consensus. Blocks propagate after PoW; stale/orphan blocks arise from latency. Bitcoin follows most accumulated work, not simply block count. 51% attacker can reorganize recent history, double-spend own txs, censor temporarily; cannot forge signatures or spend others' coins.

3 Hash Functions and Mining

3.1 SHA-256

SHA-256 accepts messages length $< 2^{64}$ bits, outputs 256 bits. Padding: append 1 bit, zeros until length $\equiv 448 \bmod 512$, append 64-bit length. Split into 512-bit blocks; compression uses message schedule $W_0, \ldots, W_{63}$, working words $a, \ldots, h$, constants, rotations/shifts, functions Ch, Maj, $\Sigma_0, \Sigma_1, \sigma_0, \sigma_1$. Avalanche: small input change changes output unpredictably. Bitcoin uses SHA256d to avoid length-extension issues and for conservative design.

3.2 Proof of Work

Valid block if SHA256d(header) $< T$. Difficulty $D = T_1/T$ where T_1 is max target. Expected hashes per valid block $\approx 2^{256}/T = D \cdot 2^{32}$ under Bitcoin convention. Difficulty retarget every 2016 blocks toward 10 min/block, bounded by protocol adjustment rules.

Nonce exhaustion: 4-byte `nNonce` gives 2^{32} tries; miners vary extranonce in coinbase, hence Merkle root. Can vary timestamp within rules; version bits signal soft forks, so should not freely abuse. `nTime` must exceed median of previous 11 blocks and not exceed network-adjusted time by more than 2h.

Mining pools: miners search shares at easier target; pool pays proportional/PPS/PPLNS. Pool centralization risks censorship and majority concentration. **Selfish mining**: strategic withholding can earn disproportionate revenue above threshold depending on network advantage γ. **Mempool lifecycle**: broadcast, validation, fee-rate ordering, eviction, replacement rules, mined or dropped. **Lightning**: payment channels use funding tx, commitment txs, HTLCs, revocation penalties for fast off-chain payments.

3.3 AsicBoost

Bitcoin header is 80 bytes, so SHA-256 processes two 64-byte chunks. Chunk 1: `nVersion` + `hashPrevBlock` + first 32 bytes Merkle root. Chunk 2: last 4 bytes Merkle root + `nTime` + `nBits` + `nNonce`. Key insight: message schedule/internal W_j calculation is independent of previous compression state $H^{(i-1)}$. If two Merkle roots share same last 4 bytes, chunk 2 is identical and miners can reuse computation. Result: about 20% speedup. Proposed by Timo Hanke and Sergio Demian Lerner.

4 Group Theory

4.1 Groups, Cyclic Groups, Fields

Group $(G, \star)$: closed, associative, identity e, inverse for every element. Abelian if $a \star b = b \star a$. Finite group order $= |G|$. Cyclic group: some generator g produces all elements $\{g, g^2, g^3, \ldots\}$ (or additive multiples). $\mathbb{Z}_n = \{0, \ldots, n-1\}$ under addition mod n is cyclic with generator 1. $\mathbb{Z}_n^* = \{i \in \mathbb{Z}_n \setminus \{0\} : \gcd(i, n) = 1\}$ under multiplication mod n is a group.

Discrete log: in cyclic group $\langle g \rangle$, given $h = g^x$, find x. Easy to compute g^x, hard to invert in selected groups. Finite field: set with addition/multiplication where nonzero elements form multiplicative group; $\mathbb{F}_p = \mathbb{Z}_p$ for prime p. Modular inverse exists iff $\gcd(a, n) = 1$; compute by extended Euclidean algorithm. Fermat: $a^{p-1} \equiv 1 \pmod p$ for $p \nmid a$. Lagrange: subgroup order divides group order; element order divides $|G|$. Quadratic residue mod prime p: $a = x^2 \bmod p$; Euler criterion $a^{(p-1)/2} \equiv 1$ for residue and -1 for non-residue.

5 Group Theory and Bitcoin ECC

5.1 Elliptic Curves

Curve over field: $E : y^2 = x^3 + ax + b$ with $4a^3 + 27b^2 \neq 0$ plus point at infinity $\mathcal{O}$. Over $\mathbb{F}_p$ use modular arithmetic. Inverse of (x, y) is $(x, -y)$. Identity $\mathcal{O}$. Point addition for $P = (x_1, y_1)$, $Q = (x_2, y_2)$:

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}, \quad x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1.$$

Doubling:

$$\lambda = \frac{3x_1^2 + a}{2y_1}, \quad x_3 = \lambda^2 - 2x_1, \quad y_3 = \lambda(x_1 - x_3) - y_1.$$

All operations mod p. Scalar multiplication by double-and-add. ECDLP: given $G, Q = xG$, find x hard.

5.2 secp256k1 and ECDSA

Bitcoin curve: $y^2 = x^3 + 7$ over prime field $p = 2^{256} - 2^{32} - 977$. Generator G, order $n =$ `FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFE BAAEDCE6 AF48A03B BFD25E8C D0364141`$_{16}$. Cofactor 1.

Keygen: private $d \in_R [1, n-1]$, public $Q = dG$. **ECDSA sign**: $z = H(m)$, choose fresh secret nonce k, $R = kG$, $r = R_x \bmod n$, $s = k^{-1}(z + rd) \bmod n$. Signature (r, s). Never reuse k: $d = (sk - z)r^{-1}$; two signatures reveal k and d. Low-s normalization prevents malleability.

Verify: reject invalid ranges; $w = s^{-1}$, $u_1 = zw$, $u_2 = rw$, $X = u_1 G + u_2 Q$; accept if $X_x \bmod n = r$. **Schnorr/Taproot**: linear signatures enable key aggregation and simpler non-malleability; Taproot combines Schnorr + Merkle script paths.

ECDSA recovery: recID 0: $x = r$, even y; 1: $x = r$, odd y; 2: $x = r + n$, even y; 3: $x = r + n$, odd y. Ethereum pre-Spurious Dragon $v = 27/28$; post-EIP-155 $v = 2 \cdot \text{chainId} + 35$ or 36.

6 Bitcoin Transactions and Script

6.1 Transaction Format

Legacy transaction: version, input count, inputs, output count, outputs, locktime. Input: previous txid, output index, scriptSig length, scriptSig, sequence. Output: value in satoshis, scriptPubKey length, scriptPubKey. Coinbase tx has special input, creates block subsidy + fees; first tx in block.

SegWit: separates witness from txid calculation; fixes third-party tx malleability. Weight = base size ×4 + witness size; virtual bytes = weight/4. P2WPKH scriptPubKey: `0 <20-byte-keyhash>`; witness: signature, pubkey.

6.2 Script

Bitcoin Script is stack-based, intentionally not Turing complete. Evaluate unlocking script then locking script; spend valid if final stack top true and no failure. Common opcodes: `OP_DUP`, `OP_HASH160`, `OP_EQUALVERIFY`, `OP_CHECKSIG`, `OP_CHECKMULTISIG`, `OP_RETURN`, `OP_CHECKLOCKTIMEVERIFY`, `OP_CHECKSEQUENCEVERIFY`.

P2PKH: scriptPubKey

```
OP_DUP OP_HASH160 <PubKeyHash> OP_EQUALVERIFY OP_CHECKSIG
```

scriptSig: `<Signature> <PublicKey>`. Steps: duplicate pubkey, hash, compare to keyhash, verify sig against transaction digest.

P2SH: locking script `OP_HASH160 <redeemScriptHash> OP_EQUAL`; spender provides redeemScript and data. Lets complex scripts be hidden until spend. **Multisig**: `OP_m <PK1> ... <PKn> OP_n OP_CHECKMULTISIG`; historical off-by-one dummy item in scriptSig. **Timelocks**: absolute locktime via `nLockTime`/CLTV; relative via `nSequence`/CSV.

7 Ethereum Overview

7.1 Platform

Ethereum = blockchain for decentralized applications: code + state on chain; txs execute code, update state, emit events/logs. History: whitepaper 2013, presale 2014, launch July 30 2015, Merge to PoS Sep 15 2022. Applications: ICOs, NFTs, DeFi.

	Bitcoin	Ethereum
State	UTXO	account/world state
Consensus	SHA256 PoW	PoS after Merge
Contracts	Script	EVM bytecode
Interval	10 min	about 12 s
Fees	sat/vB	gas, EIP-1559

Accounts: EOA controlled by private key; contract account controlled by code. Account value [nonce, balance, storageRoot, codeHash]. Address: last 20 bytes of Keccak-256(pubkey) for EOAs. Ether units: 1 ETH = 10^{18} wei; gwei = 10^9 wei.

Gas: each opcode costs gas to meter computation/storage and prevent DoS. Transaction specifies gas limit and fee parameters. Out-of-gas reverts state but gas spent is not refunded. Contract execution deterministic across nodes.

7.2 EIP-1559 Fees

Type-2 tx fields include chainId, nonce, maxPriorityFeePerGas, maxFeePerGas, gasLimit, destination, value, data, accessList, signature. Base fee adjusts per block based on gas used relative to target and is burned. Effective gas price = min(maxFee, baseFee + priorityFee). Miner/validator tip = effective - base. Sender prepays gasLimit×maxFee; unused gas and excess fee refunded. Nonce prevents replay/order ambiguity; chainId prevents cross-chain replay.

8 Ethereum Encoding and State

8.1 RLP

RLP serializes byte arrays and lists. For byte array x: if $|x| = 1$ and byte $< 0x80$, encoding is x; if $|x| < 56$, prefix $0x80 + |x|$ then x; long string prefix $0xb7 + \ell(\text{len})$, length, data. For list payload length L: short prefix $0xc0 + L$ if $L < 56$; long prefix $0xf7 + \ell(L)$ then L then payload. Integers are minimal big-endian byte arrays; zero often empty string in Ethereum encodings. BE = big-endian representation.

First byte ranges: `0x00-0x7f` single byte; `0x80-0xbf` byte array; `0xc0-0xff` list. **Worked encodings**: `0xaabbcc -> 0x83 aabbcc` since $128 + 3 = 0x83$; empty byte array $\epsilon \to$ `0x80`; byte `0x80 -> 0x81 80` because value 128 is not < 128; empty list `[] -> 0xc0`; `[[]] -> 0xc1 c0`; `[[],[[]],[[],[[]]]] -> 0xc7 c0 c1 c0 c3 c0 c1 c0`.

8.2 Merkle Patricia Trie

Ethereum authenticated key-value structure uses nibbles and Keccak hashes. Node types: branch node with 16 children + optional value; extension node for shared path; leaf node for terminal path/value. Hex-prefix encodes odd/even path length and leaf/extension flag. If RLP(node) < 32 bytes, embed; else store hash. Four major tries: state trie (accounts), storage trie per contract, transaction trie, receipts trie. Roots in block header authenticate state/txs/receipts. Bloom filters in receipts/logs support approximate event search with false positives, no false negatives.

Hex-prefix table: first nibble 0 = extension/even path; 1 = extension/odd; 2 = leaf/even; 3 = leaf/odd. Examples: leaf $[0, f, 1, c, b, 8] \to$ `20 0f 1c b8`; leaf $[f, 1, c, b, 8] \to$ `3f 1c b8`; extension $[1, 2, 3, 4, 5] \to$ `11 23 45`; extension $[0, 1, 2, 3, 4, 5] \to$ `00 01 23 45`.

9 Ethereum Transactions, EVM, Blocks

9.1 Transactions

World-state transition $\sigma_{t+1} = \Upsilon(\sigma_t, T)$. Only EOAs initiate external txs; contracts can create internal message calls during execution. Types covered: legacy, EIP-2930 access-list, EIP-1559 dynamic-fee; as of Pectra there are five total in Ethereum.

Legacy tx: nonce, gasPrice, gasLimit, to, value, data, v,r,s. Contract creation has empty `to`; created address derived from sender and nonce (or CREATE2 from deployer, salt, init-code hash). Message call invokes destination with calldata. Function selector = first 4 bytes of Keccak-256(signature), e.g. `transfer(address,uint256)`. ABI encodes static types inline in 32-byte words; dynamic types use offset then length/data padded to 32 bytes.

9.2 EVM

Stack machine, 1-byte opcodes, 32-byte words. Stack max 1024 words. Code in virtual ROM; stack and memory volatile; storage persistent per contract key-value map word→word. Memory is byte array with expanding cost (quadratic component). Current contract storage via `SLOAD/SSTORE`; other contract code via `EXTCODECOPY`; calldata via `CALLDATALOAD/CALLDATACOPY`. Call stack depth limit 1024; each call frame has own stack/memory but shared persistent state subject to revert.

Opcode families: arithmetic `ADD MUL SUB DIV MOD EXP`; comparison/bit `LT GT EQ ISZERO AND OR XOR NOT BYTE SHL SHR`; crypto `KECCAK256`; environment `ADDRESS CALLER CALLVALUE CALLDATALOAD CODESIZE GASPRICE`; block `BLOCKHASH COINBASE TIMESTAMP NUMBER PREVRANDAO GASLIMIT BASEFEE`; memory/storage `MLOAD MSTORE SLOAD SSTORE`; control `JUMP JUMPI STOP RETURN REVERT SELFDESTRUCT`; calls `CALL STATICCALL DELEGATECALL CREATE CREATE2`.

9.3 Blocks

Eth1 block = (Header, Transactions, Uncle/Ommers). Header fields: parentHash, ommersHash, beneficiary, stateRoot, transactionsRoot, receiptsRoot, logsBloom, difficulty, number, gasLimit, gasUsed, timestamp, extraData, mixHash, nonce, baseFeePerGas after London. Ommer = stale sibling of parent; rewarded in PoW because short block times. Uncle txs invalid; ommer creator got partial reward and no tx fees. GHOST chooses heaviest subtree; Eth2 uses LMD-GHOST variant.

Post-Merge execution header changes: ommersHash = hash empty list, difficulty = 0, nonce = eight zero bytes, mixHash renamed/reused as prevRandao, baseFeePerGas present. Execution client runs EVM/state; beacon client runs PoS consensus and finality.

10 Solidity and Smart Contracts

10.1 Language Essentials

Solidity: statically typed, JavaScript-like syntax, compiles to EVM bytecode. Variables: local (function, not persistent), state (contract storage), special globals. Value types: bool, uint/int sizes, address, bytes1..bytes32, enum. Reference types: arrays, structs, mappings; data locations `storage`, `memory`, `calldata`; calldata is read-only and cheaper for external inputs.

Constants/immutables: `constant` fixed at compile time, saves gas; `immutable` set once in constructor. Mappings: `mapping(K=>V)`, no iteration, key cannot be reference type, default values. Arrays: fixed/dynamic; dynamic supports push/pop/length. Structs group fields. Events write logs; indexed parameters searchable; apps listen via full node/ethers.js.

Function visibility: `public`, `external`, `internal`, `private`. Mutability: `view` reads no state mutation, `pure` reads no state, `payable` accepts ETH. `require` validates inputs/conditions and refunds remaining gas; `assert` for invariants/panic. Modifiers run code before/after function body. Inheritance via `is`; `virtual`/`override`; multiple inheritance searches parents right-to-left; state variables cannot be overridden.

10.2 Calls, Interfaces, Ether

High-level external calls use typed contract interfaces. Low-level `call` returns $(success, data)$ and forwards gas; handle return value. `delegatecall` executes callee code in caller storage context (proxy pattern risk). `receive()` handles empty calldata ETH; `fallback()` handles missing function or calldata. Ether sent to a contract without receive/fallback can fail or be stuck depending path.

Reentrancy: external call before state update lets attacker re-enter. DAO lost 3.54M ETH; Ethereum hard forked. Defenses: checks-effects-interactions (update balance before transfer), reentrancy guard modifier, pull payments, limit external calls.

ERC-20: standard fungible token interface: `name`, `symbol`, `decimals`, `totalSupply`, `balanceOf`, `transfer`, `allowance`, `approve`, `transferFrom`, `Transfer`/`Approval` events. **ERC-721**: NFTs are token IDs in contract state; media usually off-chain URI. Use OpenZeppelin implementations when possible.

Common vulnerabilities: reentrancy, integer/logic mistakes, unchecked low-level calls, bad access control,

tx.origin auth, timestamp dependence, front-running/MEV, delegatecall storage collision, upgrade initializer bugs, randomness from block variables, DoS via unbounded loops, unsafe selfdestruct assumptions.

11 JavaScript, Node, Hardhat

JavaScript created for browser interactivity; Node.js runtime runs JS outside browser and powers Hardhat tooling. Use `console.log`, `node file.js`. Variables: `let` mutable block-scoped, `const` immutable binding, avoid `var`. Types include number, bigint, string, boolean, object, undefined, null. Functions: declarations, arrows. Arrays: `push`, `map`, `filter`, `reduce`, `for...of`. Objects store key/value pairs; destructuring useful.

Promises represent future result. `async function f()` returns Promise; `await` pauses until settled. Use `try/catch`. Node modules: `require` or ES imports depending project. `npm` installs packages. `inquirer` accepts CLI input. `ethers.js`: provider reads chain, signer sends tx, contract object wraps ABI+address. Important calls: `await contract.method()`, tx response then `await tx.wait()`, parse units with `parseEther/parseUnits`.

12 Tornado Cash and zk

12.1 Mixer Protocol

Motivation: direct ETH payments reveal total balance and links. Tornado Cash = Ethereum mixer contract with fixed-denomination pools. Deposit creates commitment; withdrawal proves membership without linking deposit.

Browser generates private note: nullifier + secret (62 random bytes in slides). Commitment $C = H(\text{nullifier}, \text{secret})$ inserted into on-chain Merkle tree. User stores note. Withdrawal public inputs include Merkle root, nullifier hash, recipient, relayer, fee/refund depending version. Private witness includes note and Merkle path. Contract verifies zkSNARK, checks root known and nullifier hash unused, then pays recipient/relayer and marks nullifier spent.

Desired properties: completeness, soundness/no double spend, zero knowledge/link privacy, fixed denomination anonymity set, non-custodial contract. **OFAC**: Tornado Cash addresses sanctioned; legal/regulatory relevance.

12.2 Commitments and SNARKs

Commitment scheme: hiding + binding. Pedersen commitment $C(a, x) = xG + aH$ with unknown $\log_G H$: perfectly hiding if x random; binding under discrete-log hardness; homomorphic $C(a, x) + C(b, y) = C(a + b, x + y)$.

Pedersen proof tricks: prove C commits to 0 by signature verifiable under public key C; if $C = xG + aH$ and $a \neq 0$, this requires knowing $\log_G H$. Prove known value a by signature under $C - aH$. Prove C_1, C_2 commit to same amount by signature under $C_1 - C_2 = (x_1 - x_2)G$. Homomorphic: $C(a_1, x_1) + C(a_2, x_2) = C(a_1 + a_2, x_1 + x_2)$.

SNARK = Succinct Non-interactive Argument of Knowledge. zkSNARK adds zero knowledge. Soundness only against PPT provers (argument), proof short and one-message. Statements must be arithmetized over prime field with additions/multiplications.

Groth16: small proofs, fast verification, trusted setup circuit-specific. Ethereum verifier cost $\approx 181{,}000 + 6{,}150k$ gas for k public inputs; at 20 gwei and \$3000/ETH, about \$12 for 3 public inputs. `ecPairing` precompile at address `0x08`, cost 45,000 gas. Used by Tornado Cash/Dark Forest.

13 circom and R1CS

13.1 R1CS

R1CS expresses constraints

$$\left(\sum_i u_i a_i\right)\left(\sum_i v_i a_i\right) = \left(\sum_i w_i a_i\right)$$

where a_i are witness/signal values and (u_i, v_i, w_i) are circuit constants. Rank 1 because product of two linear combinations. Boolean bit constraint: $b(1 - b) = 0$. AND: $c = ab$. OR: $c = a + b - ab$. XOR: $c = a + b - 2ab$. NOT: $c = 1 - a$.

13.2 circom Signals and Operators

circom = circuit compiler for zk statements, uses Groth16 toolchain. Signals are immutable field elements; components are subcircuits; inputs private by default; public signals declared in main component. `===` adds equality constraint. `<--` assigns witness/advice without constraint. `<==` assigns and constrains when expression is quadratic-valid. Only quadratic constraints allowed; non-quadratic computations require advice plus constraints.

Patterns: arrays of signals/components; `var` for mutable compile-time variables. Multiplexer: if selector bit s, output $(1 - s)c_0 + sc_1$. Zero check uses advice inverse: if input nonzero, constrain $in \cdot inv = 1$ and output 0; if zero, output 1 with constraints preventing false positive. Bit decomposition: constrain each bit boolean and $\sum 2^i b_i = in$; range proof if fits n bits. Comparator: compare via bit decomposition of shifted difference; circom default field fits about 253-bit safe arithmetic.

Tornado circuits: Merkle tree path checker + withdrawal checker; public root/nullifierHash/recipient/relayer/fee, private nullifier/secret/path.

13.3 Key circom Circuits

IsZero: `inv <-- in!=0 ? 1/in : 0; out <== -in*inv + 1; in*out === 0`. **Num2Bits(n)**: `out[i] <-- (in >> i) & 1;` constrain `out[i]*(out[i]-1)===0;` constrain $\sum 2^i$`out[i] = in`. **LessThan(n)**: `n2b.in <== in[0]+(1<<n)-in[1]; out <== 1-n2b.out[n];` output 1 iff `in[0] < in[1]`. **Mux**: `s*(s-1)===0; out <== (c[1]-c[0])*s+c[0]`.

14 Monero

14.1 Privacy Model

Monero launched Apr 2014, privacy-oriented, CryptoNote-based. Hides amounts, uses one-time addresses, ring signatures/linkable ring signatures to weaken chain analysis. Popular in cryptojacking/ransomware because privacy + CPU/GPU mining history.

Stealth/one-time address: Bob has private (x_1, x_2), public $(P_1 = x_1 G, P_2 = x_2 G)$. Alice picks r, computes $P = H_s(rP_1)G + P_2$, publishes P and $R = rG$. Bob scans: $P' = H_s(x_1 R)G + P_2$; if equal, spend key $x = H_s(x_1 R) + x_2$ satisfies $P = xG$. Tracking key (x_1, P_2) lets third party detect incoming payments without spend key.

One-time address isolated: setup Bob private (x_1, x_2), public $(P_1 = x_1 G, P_2 = x_2 G)$. Send: Alice random r, output $P = H_s(rP_1)G + P_2$, publish $P, R = rG$. Receive: Bob tests $P' = H_s(x_1 R)G + P_2$; if $P' = P$, spend key $x = H_s(x_1 R) + x_2$. Tracking key (x_1, P_2) audits incoming only.

14.2 Ring and Linkable Ring Signatures

Ring signature proves signer knows one private key among $P_0, \ldots, P_{n-1}$ without revealing which. Basic flow: choose random α, s_i for $i \neq j$; set $L_j = \alpha G$, challenge chain $c_{j+1} = H_s(m, L_j)$, compute $L_i = s_i G + c_i P_i$ around ring, set $s_j = \alpha - c_j x_j$; signature $(c_0, s_0, \ldots, s_{n-1})$ verifies if challenge closes.

Linkable LSAG adds key image $I = x_j H_p(P_j)$. Compute both $L_i = s_i G + c_i P_i$ and $R_i = s_i H_p(P_i) + c_i I$; signature includes I. Duplicate key images are rejected, preventing double spend while preserving signer ambiguity. Source address obfuscation: spender inserts true one-time address among decoys; outsider cannot identify true input or even know membership cleanly.

LSAG steps: key image $I = x_j H_p(P_j)$. Sign: pick α, s_i for $i \neq j$; $L_j = \alpha G$, $R_j = \alpha H_p(P_j)$, $c_{j+1} = H_s(m, L_j, R_j)$. For $i \neq j$, $L_i = s_i G + c_i P_i$, $R_i = s_i H_p(P_i) + c_i I$, $c_{i+1} = H_s(m, L_i, R_i)$. Set $s_j = \alpha - c_j x_j$. Signature $\sigma = (I, c_0, s_0, \ldots, s_{n-1})$. Verify recomputes chain and checks last hash returns c_0; duplicate I rejected.

14.3 Confidential Transactions

Amounts hidden with Pedersen commitments $C(a, x) = xG + aH$. Balance condition over commitments:

$$\sum C_i^{in} - \sum C_j^{out} - C(f, 0) = C(0, x)$$

prove zero commitment by signature with commitment as public key / knowledge of blinding. Modular arithmetic alone unsafe: attacker could output $p - 4$ amount if no range proof. Range proofs prove committed amounts in valid range (e.g., 64-bit). Naive bit proof: commit to $a_i 2^i$ and prove each bit is 0/1 with ring signatures; Bulletproofs reduce proof size to $O(\log n)$.

Range proof construction: modular balance alone permits commitment to $p - 4$ as a huge hidden amount. Naive proof ring-signs over $\{C - iH : 0 \leq i < 2^{64}\}$, size $O(2^{64})$. Better: decompose $a = \sum_{i=0}^{63} a_i 2^i$, $a_i \in \{0, 1\}$; create $C_i = x_i G + a_i 2^i H$; ring sig over $\{C_i, C_i - 2^i H\}$ proves bit 0/1. If $x = \sum x_i$, then $C(a, x) = \sum C_i$. Bulletproofs give $O(\log n)$ proof size for n-bit range.

RingCT: each output has one-time address P and Pedersen commitment C. For m inputs and ring size n, spender builds matrix of key vectors/commitments; MLSAG proves knowledge of all private keys in one column and balance with fees/outputs, while key images prevent double spend.

15 Consensus Theory

15.1 SMR and Models

Consensus protocols keep nodes in sync despite unreliable network and faults. State Machine Replication: clients submit txs; each node keeps append-only ordered history; if same initial state and same order, same final state.

Requirements: **consistency/safety**: honest nodes agree on same history; **liveness**: every valid tx submitted eventually included.

Network models: synchronous = known message delay bound/global rounds; asynchronous = no timing guarantees; partially synchronous = bound Δ exists after unknown GST. Faults: crash (stop), omission (drop sends/receives), Byzantine (arbitrary but cannot forge signatures). Permissioned = fixed known validators/PKI; permissionless requires Sybil resistance (PoW/PoS).

Model	Timing
Synchronous	global clock, delays $\leq \Delta$ per step, same timestep
Asynchronous	no clock/bound; messages eventually arrive
Partial sync	arbitrary until unknown GST, then delays $\leq \Delta$

15.2 Byzantine Broadcast

Single sender with input v^*. Properties: termination (honest halt), agreement (same output), validity (if sender honest, output v^*). SMR from BB: each epoch leader proposes list of pending txs; run BB; append output. Liveness usually for txs reaching honest leaders/nodes.

For $f = 1$ cross-checking: sender signs value to all; recipients sign/forward; if see one valid value output it, conflicts handled by default depending protocol.

Dolev-Strong (PKI synchronous): tolerate $f < n$ Byzantine in $f + 1$ rounds using signature chains. In round j, forward values with valid chain length j and distinct signers; accept only values with valid length $f + 1$ at end. Signatures prevent late equivocation.

Dolev-Strong exact: convincing message for value v at time t references v, is signed by sender, and signed by at least $t - 1$ other distinct nodes. Time 0 sender sends v^* with signature to all. Times $t = 1$ to $f + 1$: if convinced of new v, sign and forward. Final output: exactly one convinced value v gives v, otherwise $\perp$. Tolerates any f Byzantine in synchronous PKI.

Bounds: without PKI, Byzantine agreement in synchrony needs $n > 3f$ (FLM-style). Asynchronous deterministic consensus impossible for $n \geq 2$, $f = 1$ (FLP). Partial synchrony BFT possible iff $n > 3f$ for Byzantine faults.

FLP: no deterministic async protocol satisfies termination+agreement+validity with $f = 1$. **FLM**: without PKI, if $f \geq n/3$, no Byzantine broadcast. **CAP**: cannot have consistency+availability+partition tolerance; Tendermint/PBFT sacrifice liveness during partition, Bitcoin sacrifices consistency via reorgs.

16 Tendermint

Tendermint = partially synchronous BFT consensus, permissioned/PKI, consistency always and eventual post-GST liveness if $f < n/3$. Real deployments in Cosmos ecosystem; votes may be stake-weighted, but slides describe one vote/node.

Rounds: known Δ after GST. Round length 4Δ: propose, prevote/stage-1, precommit/stage-2, commit/timeout. Unique rotating leader per round. Quorum certificate (QC) = signatures from $> 2n/3$ validators for same block/round/stage.

Phase timing: round r starts at $t = 4\Delta r$. Phase 1 $t = 4\Delta r$: leader broadcasts (B_l, Q_l). Phase 2 $t = 4\Delta r + \Delta$: nodes cast stage-1 vote for B_i. Phase 3 $t = 4\Delta r + 2\Delta$: if $> 2n/3$ stage-1 votes, cast stage-2 vote and form stage-1 QC. Phase 4 $t = 4\Delta r + 3\Delta$: if $> 2n/3$ stage-2 votes, commit, increment height, form stage-2 QC. QC order: non-null > null; later round > earlier; stage-2 > stage-1 in same round.

Local state: locked block/value and lock round; valid block/round for leader proposal. Protocol idea: leader proposes block, nodes prevote if proposal valid and compatible with lock; if receive $> 2/3$ prevotes, precommit and lock; if receive $> 2/3$ precommits, commit. If timeout, move to next round. Safety: two conflicting commits would require two $> 2/3$ quorums intersecting in $> n/3$ nodes, at least one honest double-voting, impossible with locking rules. Liveness after GST: eventually honest leader with compatible valid value and sufficient timeout gets proposal/votes through.

17 Ethereum Proof of Stake

17.1 Validators, Slots, Committees

Merge on Sep 15 2022 moved Ethereum to PoS, reducing power from 5.13 GW to 2.62 MW in slides (99.95%). Validators deposit 32 ETH in deposit contract; can later withdraw deposit/rewards. Slot = 12 s, one block proposer selected; epoch = 32 slots = 6.4 min. Validators assigned to committees per slot to reduce communication; one active validator proposes, committees attest.

BLS signatures: groups G_1, G_2 of prime order p with pairing $e : G_1 \times G_2 \to G_T$; bilinear $e(g_1^\alpha, g_2^\beta) = e(g_1, g_2)^{\alpha\beta}$. Secret x, public g_1^x. Sign $\sigma = H(m)^x$ with $H(m) \in G_2$. Verify $e(g_1, \sigma) = e(g_1^x, H(m))$. Same-message aggregate: $\sigma_{\text{agg}} = \prod \sigma_i$, $pk_{\text{agg}} = \prod pk_i$, verify aggregate under pk_{agg}. Enables compact committee attestations.

Randomness: RANDAO contributions produce on-chain randomness; committee assignment shuffles validators. Security needs unpredictability/unbiasability enough to prevent adversarial committee capture.

17.2 Fork Choice and Finality

LMD-GHOST: Latest Message Driven Greediest Heaviest Observed SubTree. Starting at genesis/finalized checkpoint, at each fork choose child with greatest total stake from latest attestations of validators; repeat. Uses only latest vote per validator to avoid vote duplication weight.

Casper FFG: checkpoints every epoch. Vote is source checkpoint $\to$ target checkpoint. Justification: supermajority link $s \to t$ ($> 2/3$ stake) with s justified $\Rightarrow t$ justified. Finalization: justified $c_1 \to c_2$ supermajority where c_2 is direct child of $c_1 \Rightarrow c_1$ finalized. Rule 1 no double vote: validator must not publish $s_1 \to t_1$ and $s_2 \to t_2$ where $h(t_1) = h(t_2)$. Rule 2 no surround vote: must not publish votes where $h(s_1) < h(s_2) < h(t_2) < h(t_1)$. If $< 1/3$ Byzantine, conflicting checkpoints cannot both finalize without slashable votes.

18 Quick Tables

18.1 Impossibility and Models

Result	Meaning
FLP	async + deterministic + $f \geq 1$ gives no termination+agreement+validity
FLM	sync + no PKI + $f \geq n/3$ gives no Byzantine broadcast
Partial sync BFT	possible iff $f < n/3$; Tendermint/PBFT achieve this

18.2 Ethereum Tx Types and Gas

Type	Fields
0 legacy	nonce, gasPrice, gasLimit, to, value, data, v, r, s
1 EIP-2930	adds chainId, accessList, signatureYParity
2 EIP-1559	maxPriorityFeePerGas + maxFeePerGas replace gasPrice
Pectra	as of May 2025: 5 total tx types

Gas item	Cost
Tx base	21,000
Calldata	zero 4, nonzero 16
SLOAD	cold 2100, warm 100
CALL address	cold 2600, warm 100
CREATE	32,000 minimum
KECCAK256	30 + 6 per word
ecPairing	45,000 gas

18.3 Privacy and Scripts

Monero term	Meaning
One-time address	hides receiver, fresh each tx
Ring signature	hides sender among decoys
Key image	$I = xH_p(P)$ links double spends only
Pedersen	hides amount; homomorphic
Range proof	amount valid without revealing
Bulletproofs	$O(\log n)$ range proof size
Tracking key	(x_1, P_2) audits incoming
MLSAG	multi-layer linkable spontaneous anonymous group sig

	Bitcoin	Ethereum
Script	stack-based, not Turing complete, stateless	EVM bytecode, gas-bounded Turing complete, stateful, 1024-word stack

Commitment	Property
Hiding	random blinding reveals nothing
Binding	cannot open differently unless know $\log_G H$
Homomorphic	$C(a, x) + C(b, y) = C(a + b, x + y)$

18.4 Header and Transaction Fields

Item	Must remember
Bitcoin header	version, prev hash, Merkle root, time, bits, nonce; hash = SHA256d
Bitcoin tx	inputs spend UTXOs; outputs create UTXOs; fee = in - out
Ethereum account	nonce, balance, storageRoot, codeHash
Eth tx	nonce, gas fields, to/creation, value, data, access list, signature
Eth header	parent, state/tx/receipt roots, logsBloom, beneficiary, gas, time, prevRandao/baseFee

18.5 High-Yield Formulas

- Hash collision work $2^{n/2}$; preimage work 2^n.
- Bitcoin PoW valid iff SHA256d(header)$< T$; expected hashes $\approx D2^{32}$.
- ECDSA: $s = k^{-1}(z + rd)$; verify $X = (zs^{-1})G + (rs^{-1})Q$.
- Ethereum fee: effective price $= \min(\text{maxFee}, \text{baseFee} + \text{tip})$; base fee burned.
- Pedersen: $C(a, x) = xG + aH$; additive homomorphism.
- R1CS: $(u \cdot a)(v \cdot a) = w \cdot a$.
- Tendermint/Ethereum BFT thresholds: $> 2/3$ quorum; safety if Byzantine stake/nodes $< 1/3$.

18.6 Common Confusions

- Bitcoin addresses are hashes/encodings of public keys, not public keys themselves until spent in P2PKH.
- Most-work chain differs from longest-by-count chain.
- SegWit fixes txid malleability by moving signatures to witness.
- Ethereum contracts cannot initiate top-level txs; they execute because an EOA tx or another call invoked them.
- `view`/`pure` are Solidity promises enforced for external calls/compilation; gas still used if called inside tx.
- Logs/events are not contract-readable state; they are for off-chain indexing.
- A zkSNARK proof hides witness but public inputs are public; nullifier prevents double withdrawal.
- Ring signatures hide which input was spent; key images still link double spends.
- FLP is about deterministic async consensus; practical BFT gets liveness by synchrony/partial synchrony assumptions.

19 Exam Templates

19.1 Calculation Checklists

EC over $\mathbb{F}_p$: verify point by comparing y^2 and $x^3 + ax + b \bmod p$; inverse $(x, -y \bmod p)$; if $P = Q$ use doubling slope; if $P \neq Q$ use add slope; compute modular inverse by trial/Euclid; reduce $\lambda, x_3, y_3 \bmod p$; if denominator 0 then result $\mathcal{O}$. **Group order**: for each x, compute RHS; count 0 roots if non-residue, 1 if RHS=0, 2 if residue; add $\mathcal{O}$.
RLP: convert item to bytes; single byte $< 0x80$ encodes itself; short string prefix $0x80 + \ell$; short list prefix $0xc0+$ payload length; nested lists use already-RLP-encoded children as payload. **HP**: decide leaf/ext and odd/even; even uses flag nibble then 0 pad; odd packs flag nibble with first path nibble; pack remaining nibbles two per byte.
Gas/fees: tx cost $= 21000 + \text{calldata} + EVM/storage$; EIP-1559 effective price $= \min(\text{maxFee}, \text{baseFee} + \text{tip})$; burned $= \text{baseFee} \cdot \text{gasUsed}$; validator tip $= (\text{effective} - \text{baseFee}) \cdot \text{gasUsed}$; refund unused gas and unused max fee.

19.2 Protocol Trace Templates

Casper: for each vote write source height, target height, voter/stake. Justified iff source justified and $> 2/3$ link. Finalized iff justified parent has justified direct child. Slash if same target height or surround inequality $h(s_1) < h(s_2) < h(t_2) < h(t_1)$.
Tendermint: per round write leader proposal (B, Q), prevotes/stage-1, precommits/stage-2, lock state. Commit requires $> 2n/3$ stage-2 votes. Safety proof always uses quorum intersection: two $> 2n/3$ quorums intersect in $> n/3$, so at least one honest validator would need to violate lock/vote rule.
Dolev-Strong: at time t only accept chains with sender sig $+ t - 1$ distinct extra sigs; sign/forward each value once; at final time output sole convincing value else $\perp$. Late messages with too-short chains cannot convince.

19.3 circom Design Recipes

Always: signals are field elements; every Boolean needs $b(b-1) = 0$; every division/comparison/bit extraction usually needs advice plus constraints. Never trust `<--` alone; pair it with `===`/`<==`. Split degree > 2 expressions into temporaries.
Range check $x < 2^n$: run `Num2Bits(n)` and constrain reconstruction. **Equality**: constrain $a - b = 0$. **Nonzero**: advice inverse inv, constrain $x \cdot inv = 1$. **Select**: $out = (1 - s)a + sb$ with Boolean s. **Merkle path**: at each level use selector bit to order left/right, hash, constrain final root public.

19.4 Solidity Bug Checklist

Bug	Exam answer/fix
Reentrancy	external call before state update; fix checks-effects-interactions or guard
Access control	missing `onlyOwner`/role; restrict privileged state changes
Unchecked call	low-level `call` return ignored; require success and decode data
`tx.origin` auth	phishing via intermediate contract; use `msg.sender`
Delegatecall	callee writes caller storage; align storage and restrict implementation
Bad randomness	block vars manipulable/predictable; use commit-reveal/VRF
Unbounded loop	gas DoS over growing array; use pagination/pull pattern
Front-running	public mempool ordering; commit-reveal, slippage/deadline, checks

19.5 Smart Contract Writing Templates

Solidity mental model: a contract is like a class with persistent storage on-chain. State variables live forever; functions change/read them; `msg.sender` is caller; `msg.value` is ETH sent; `require(cond,"msg")` stops if false and reverts.
Basic syntax: every statement ends with `;`. Code blocks use `{...}`. Comments use `//`. Types first, name second: `uint amount; address user; bool ok; string memory s;`. Function shape:

```
function name(type arg) visibility modifier returns(type) {
  // body
}
```

Visibility: `public`=callable inside/outside; `external`=outside only; `internal`=contract/children; `private`=this contract.
Mutability: `view`=reads state; `pure`=reads no state; `payable`=can receive ETH.
Storage building blocks: `uint public x;` auto-creates getter. `mapping(address=>uint) public bal;` maps address to number. `address public owner;` stores admin. `event E(...);` records log for frontend/exam answer. `modifier m(){...;...}` wraps functions.
Exam skeleton:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
contract X {
  address public owner;
  event Done(address indexed user,uint amount);
  error NotOwner(); error BadAmount();
  constructor(){ owner = msg.sender; }
  modifier onlyOwner(){ if(msg.sender!=owner) revert NotOwner(); _; }
}
```

Tiny storage contract:

```
contract Store {
  uint public value;
  event Changed(uint oldValue, uint newValue);
  function set(uint v) external {
    emit Changed(value, v);
    value = v;
  }
  function get() external view returns(uint) {
    return value;
  }
}
```

Owner-only pattern:

```
address public owner;
constructor(){ owner = msg.sender; }
modifier onlyOwner(){
  require(msg.sender == owner, "NOT_OWNER");
  _;
}
function setOwner(address n) external onlyOwner {
  require(n != address(0), "ZERO");
  owner = n;
}
```

Checks-effects-interactions withdraw:

```
mapping(address=>uint) public bal;
function deposit() external payable {
  bal[msg.sender] += msg.value;
}
function withdraw(uint a) external {
  if (bal[msg.sender] < a) revert BadAmount();
  bal[msg.sender] -= a;
  (bool ok,) = msg.sender.call{value:a}("");
  require(ok, "ETH_SEND_FAIL");
}
```

Reentrancy guard:

```
bool private locked;
modifier nonReentrant(){
  require(!locked,"REENTRANT");
  locked=true; _; locked=false;
}
```

Minimal payable vault:

```
contract Vault {
  mapping(address=>uint) public bal;
  bool locked;
  modifier nonReentrant(){ require(!locked); locked=true; _; locked=false;}
  receive() external payable { bal[msg.sender] += msg.value; }
  function deposit() external payable { bal[msg.sender] += msg.value; }
  function withdraw(uint a) external nonReentrant {
    require(bal[msg.sender] >= a, "BAL");
    bal[msg.sender] -= a;
    (bool ok,) = msg.sender.call{value:a}("");
    require(ok, "SEND");
  }
}
```

ERC-20 style transfer:

```
mapping(address=>uint) public balanceOf;
event Transfer(address indexed from,address indexed to,uint value);
function transfer(address to,uint amount) external returns(bool){
  require(to != address(0), "ZERO_TO");
  require(balanceOf[msg.sender] >= amount, "BAL");
  balanceOf[msg.sender] -= amount;
  balanceOf[to] += amount;
  emit Transfer(msg.sender,to,amount);
  return true;
}
```

ERC-20 approve/transferFrom skeleton:

```
mapping(address=>mapping(address=>uint)) public allowance;
event Approval(address indexed owner,address indexed spender,uint value);
function approve(address spender,uint amount) external returns(bool){
  allowance[msg.sender][spender] = amount;
  emit Approval(msg.sender, spender, amount);
  return true;
}
function transferFrom(address from,address to,uint amount) external returns(bool){
  require(balanceOf[from] >= amount, "BAL");
  require(allowance[from][msg.sender] >= amount, "ALLOW");
  allowance[from][msg.sender] -= amount;
  balanceOf[from] -= amount; balanceOf[to] += amount;
  emit Transfer(from, to, amount);
  return true;
}
```

Allowance pattern: `approve(spender, amount)` sets allowance and emits `Approval`; `transferFrom(from,to,amount)` checks allowance and balance, decreases allowance unless max uint, moves balances, emits `Transfer`. Race warning: nonzero-to-nonzero approve can be front-run; prefer increase/decrease allowance or set 0 first.
What to write when stuck: 1. declare state variables; 2. declare events/errors; 3. constructor sets owner or initial supply; 4. each function starts with `require`; 5. update storage; 6. emit event; 7. do external call last.
Receive/fallback: `receive() external payable {}` handles empty calldata ETH; `fallback() external payable {}` handles unknown selector or calldata. **Access control**: use `onlyOwner` for admin setters, pausing, minting, withdrawals. **Events**: emit after state changes; index searchable fields. **Storage vs memory**: mappings/contract state in storage; external array/string args often `calldata`; temporary structs/arrays `memory`. **Exam checklist**: validate inputs, enforce permissions, update state before external calls, handle return values, emit events, avoid unbounded loops, avoid `tx.origin`, avoid block randomness.

19.6 Monero and Pedersen Answer Skeleton

One-time address trace: write Bob public keys; Alice random r; compute $P = H_s(rP_1)G + P_2$, publish $R = rG$; Bob tests $H_s(x_1R)G + P_2$; spend key $H_s(x_1R) + x_2$. **LSAG trace**: compute/recompute (L_i, R_i, c_{i+1}) around ring and check closure to c_0; duplicate key image means double spend. **Balance proof**: subtract output commitments and fee from input commitments; if amounts balance, difference is pure xG, so signature under difference proves knowledge of blinding.

20 Worked Examples

20.1 EC Arithmetic over $\mathbb{F}_7$

Curve $y^2 = x^3 + x + 6$. Check $(2, 3)$: $3^2 = 9 \equiv 2$, $2^3 + 2 + 6 = 16 \equiv 2 \pmod 7$. Negation: $-(2, 3) = (2, -3) = (2, 4)$. Doubling:

$$\lambda = \frac{3 \cdot 4 + 1}{2 \cdot 3} = \frac{13}{6}, \quad 6^{-1} = 6, \quad \lambda = 13 \cdot 6 \equiv 1.$$

$x_3 = 1^2 - 2 - 2 = -3 \equiv 4$; $y_3 = 1(2 - 4) - 3 = -5 \equiv 2$. Result $2(2, 3) = (4, 2)$. Group order pattern: enumerate $x = 0, \ldots, 6$, compute RHS, count y roots for quadratic residues, add point at infinity.

20.2 EC List All Points

Example $E : y^2 = x^3 + x + 6$ over $\mathbb{F}_7$. First square table: $y = 0, 1, 2, 3, 4, 5, 6$ gives $y^2 = \{0, 1, 4, 2, 2, 4, 1\}$, so residues map $0 : \{0\}$, $1 : \{1, 6\}$, $2 : \{3, 4\}$, $4 : \{2, 5\}$. Now compute RHS:

x	0	1	2	3	4	5	6
$x^3 + x + 6 \bmod 7$	6	1	2	1	4	3	4

Match RHS to square table: $x = 0$ none; $x = 1 \to y = 1, 6$; $x = 2 \to y = 3, 4$; $x = 3 \to y = 1, 6$; $x = 4 \to y = 2, 5$; $x = 5$ none; $x = 6 \to y = 2, 5$. Points:

$$(1, 1), (1, 6), (2, 3), (2, 4), (3, 1), (3, 6), (4, 2), (4, 5), (6, 2), (6, 5), \mathcal{O}$$

Total $|E(\mathbb{F}_7)| = 11$. Exam trick: always add point at infinity $\mathcal{O}$.

20.3 RLP Examples

`"dog"`: bytes `64 6f 67`, length 3; prefix $128 + 3 = 131 =$ `0x83`; result `83 64 6f 67`. `["cat","dog"]`: `cat -> 83 63 61 74`, `dog -> 83 64 6f 67`; payload 8 bytes; list prefix $192 + 8 = 200 =$ `0xc8`; result `c8 83 63 61 74 83 64 6f 67`.

20.4 Pedersen Balance

Inputs $3, 5, 2$ with blinds x_1, x_2, x_3; outputs $4, 5$ with blinds y_1, y_2; fee $f = 1$ blind 0. Balance $3 + 5 + 2 = 4 + 5 + 1 = 10$. Commitment check:

$$C(3, x_1) + C(5, x_2) + C(2, x_3) - C(4, y_1) - C(5, y_2) - C(1, 0) = C(0,$$

where $x = x_1 + x_2 + x_3 - y_1 - y_2$. A signature verifiable under this difference as public key proves balance.

20.5 Ring Signature Trace

Ring size 3, signer index $j = 1$, key image $I = x_1 H_p(P_1)$. Verify: $L_0 = s_0 G + c_0 P_0$, $R_0 = s_0 H_p(P_0) + c_0 I$, $c_1 = H_s(m, L_0, R_0)$; $L_1 = s_1 G + c_1 P_1$, $R_1 = s_1 H_p(P_1) + c_1 I$, $c_2 = H_s(m, L_1, R_1)$; $L_2 = s_2 G + c_2 P_2$, $R_2 = s_2 H_p(P_2) + c_2 I$; check $H_s(m, L_2, R_2) = c_0$.

20.6 HP Encoding Examples

Extension path $[1, 2, 3, 4, 5]$ odd: first nibble 1; first byte `0x11`; remaining `23 45`; result `11 23 45`. Leaf path $[0, f, 1, c, b, 8]$ even: first nibble 2, padding 0; first byte `20`; remaining `0f 1c b8`; result `20 0f 1c b8`.

20.7 circom R1CS Trace

Circuit `out = in1*in2*in2` is cubic, so split: `tmp <== in1*in2` then `out <== tmp*in2`; both quadratic. AND with $a = 1, b = 0$: $a(1 - a) = 1 \cdot 0 = 0$, $b(1 - b) = 0 \cdot 1 = 0$, output $ab = 0$.

20.8 Casper Trace

Epochs $N, N + 1, N + 2$. If C_N justified and $> 2/3$ vote $C_N \to C_{N+1}$, then C_{N+1} justified and C_N finalized by direct-child rule. If $> 2/3$ then vote $C_{N+1} \to C_{N+2}$, then C_{N+2} justified and C_{N+1} finalized. Slash example: votes $C_2 \to C_5$ and $C_3 \to C_4$ satisfy $2 < 3 < 4 < 5$, a surround vote.

20.9 Tendermint Consistency

Round r: $> n/3$ honest set S casts stage-2 votes for B^*. Their Q_i is stage-1 QC for $(h, r, 1)$ supporting B^*. Any later leader must use newest QC; nodes in S vote only for blocks with QC at least as recent as Q_i. All round-r QCs support B^*, so S only votes B^*; no $B \neq B^*$ can get $> 2n/3$ because $|S| > n/3$.

20.10 ECDSA Nonce Reuse

Same k: $s_1 = k^{-1}(z_1 + rd)$, $s_2 = k^{-1}(z_2 + rd)$. Subtract: $s_1 - s_2 = k^{-1}(z_1 - z_2)$. Thus

$$k = (z_1 - z_2)(s_1 - s_2)^{-1} \bmod n, \quad d = (s_1 k - z_1) r^{-1} \bmod n.$$

Private key recovered; never reuse k.

20.11 Tornado Withdrawal Statement

Public inputs: Merkle root, nullifierHash, recipient, relayer, fee. Private witness: nullifier, secret, Merkle path. Circuit proves commitment = Pedersen(nullifier, secret), commitment is leaf under root, and nullifierHash = Pedersen(nullifier). Soundness: only depositor knows note. Privacy: commitment hidden, so deposit not linked. Anti-double-spend: nullifierHash stored on-chain.

21 Mini Drills with Answers

21.1 Compute Drills

EC: if denominator is $d \bmod p$, find d^{-1} first; if $y = 0$ when doubling, tangent vertical so $2P = \mathcal{O}$. **Nonce reuse**: same r usually means same k; compute $k = (z_1 - z_2)/(s_1 - s_2)$ then $d = (s_1 k - z_1)/r$. **RLP**: `0x7f -> 7f`, `0x00 -> 00`, `0x01 -> 01`, `0x8180` decodes to byte string `80`. **HP**: leaf odd path starts with flag nibble 3; extension even starts `00`.

21.2 Trace Drills

Casper: votes $A_1 \to A_4$ and $A_2 \to A_5$ are not surround because targets not nested; same target epoch is double vote even if sources differ. **Tendermint**: if a node is locked on B at round r, it may vote for different B' only if proposal carries QC newer than its lock. **Dolev-Strong**: a value first appearing too late cannot collect enough distinct signatures by final round.

21.3 Proof Hooks

Quorum intersection: two sets of size $> 2n/3$ intersect in $> n/3$ nodes; since at most $n/3$ Byzantine, at least one honest is in intersection. Use this for Tendermint/PBFT/Casper safety. **Pedersen binding**: if $xG + aH = x'G + a'H$ with $a \neq a'$, then $H = ((x - x')/(a' - a))G$, revealing $\log_G H$. **Ring privacy**: verifier sees a closed challenge cycle, not which response was programmed using secret key.

21.4 Solidity Micro-Snippets

Checks-effects-interactions: check balance/permission; update storage; then external call. **Guard**: `require(!locked); locked=true; _; locked=false;` in modifier. **ERC20 transfer invariant**: require sender balance $\geq amount$; decrement sender; increment receiver; emit `Transfer`; return true. **Approve race**: changing nonzero allowance to another nonzero allowance can be front-run; set to 0 first or use increase/decrease allowance.

21.5 MPT and Ethereum State

MPT proof: decode RLP node; use path nibbles; branch consumes one nibble and chooses child; extension consumes shared prefix; leaf must match remaining path and contains value. If encoded child < 32 bytes it is inline RLP; otherwise look up by Keccak hash. **Receipt**: status/root, cumulativeGasUsed, logsBloom, logs. **Event log**: address + topics + data; topic0 is Keccak event signature unless anonymous.

21.6 BLS and Aggregation Pitfalls

Same-message aggregation verifies with aggregated public key. Different-message aggregation needs product of pairings $\prod e(pk_i, H(m_i))$; avoid rogue-key attack by proof-of-possession or coefficient weighting. Pairing bilinearity is the whole trick: exponents multiply inside target group.

21.7 Tornado and Privacy Edge Cases

Anonymity set is only deposits of same denomination/tree/root. Relayer hides withdrawal gas payer but recipient is public. Nullifier hash must be public to stop double withdrawal; nullifier itself remains private. If user withdraws to linked address, zk proof still valid but privacy lost off-chain.

22 Source Coverage Checklist

- `HashFunctions.pdf`: hash definitions, SHA-256, collision/preimage, hash pointers.
- `GroupTheory.pdf`: groups, cyclic groups, modular arithmetic, finite fields, Lagrange/QR.
- `BitcoinIntroduction.pdf`: Bitcoin workflow, decentralization, P2P, blocks, UTXO.
- `BitcoinECC.pdf`: EC group law, secp256k1, keygen, ECDSA/Schnorr.
- `BitcoinTransactions.pdf`: transaction format, scripts, P2PKH/P2SH/multisig/SegWit/timelocks/coinbase.
- `MiningMisc.pdf`: nonce space, extranonce, difficulty, mining pools, selfish mining, mempool, Lightning.
- `EthereumIntroduction.pdf`: Ethereum history, accounts, gas, smart contracts, ERC-20.
- `EthereumDataEncoding.pdf`: RLP, MPT, HP encoding, tries, Keccak, blooms.
- `EthereumTransactions.pdf`: world state, tx types, EIP-1559, signing, execution, contract calls.
- `EVM.pdf`: stack/memory/storage/code, opcodes, gas, call frames.
- `EthereumBlocks.pdf`: Eth1/Eth2 headers, ommers, GHOST, Merge fields.
- `SolidityTutorial.pdf`: Solidity syntax/types/functions/events/inheritance/calls/reentrancy/ERC20/ERC721.
- `JavascriptTutorial.pdf`: JS/Node essentials, promises, CLI, ethers.js.
- `TornadoCash.pdf`: mixer protocol, commitments, zkSNARK withdrawal, Groth16, OFAC.
- `circom.pdf`: R1CS, boolean gates, signals/operators, mux, zero check, bit decomposition, comparator, Tornado circuits.
- `Consensus.pdf`: SMR, BB, Dolev-Strong, synchrony/FLP/thresholds.
- `Tendermint.pdf`: rounds, leaders, two-stage voting, locks, consistency/liveness.
- `EthereumConsensus.pdf`: deposits, slots/epochs, committees, BLS, RANDAO, LMD-GHOST, Casper FFG, slashing.
- `Monero.pdf`: stealth addresses, ring/LSAG, key images, Pedersen commitments, range proofs, RingCT.
- `blockchain_notes.pdf` and `EE465_quiz_cheatsheet.pdf`: integrated throughout as course-wide summaries.